IBM Z DATATHON 2026 • TECHNICAL SOLUTION & DESIGN DOCUMENT

ScamGuard-Z

An Explainable Risk-Scoring and Safety Warning Prototype for Mitigating Social Engineering and Coercion-Related Transaction Fraud

Selected Track:	**Theme 1: Real-Time AI for Critical Decisions**
Cross-Cutting Focus:	**Theme 2: AI Secured — Innovation Without Exposure**
Target Architecture:	IBM Z / LinuxONE Community Cloud (Target for Exploration & Validation)
Project Team Scope:	Six-Member Undergraduate Team (Beginner-Friendly, Realistic Implementation)
Project Status:	Technical Design Specification & Prototype Architecture

Core Problem Statement

Victims of social engineering scams may authorise fraudulent bank transfers themselves under pressure or deception. Traditional transaction checks may not identify these coercion-related risks, allowing victims to lose their savings. ScamGuard-Z analyses simulated transactions, identifies suspicious patterns, explains its risk assessment, and displays an educational safety warning encouraging users to pause and verify the recipient.

Table of Contents

1. Executive Summary — Page 2
2. Problem Statement & Project Objectives — Page 2
3. Proposed Solution & Core Prototype Features — Page 3
4. System Architecture & Workflow — Page 4
5. Technology Stack & IBM Z / LinuxONE Exploration — Page 5
6. Data Strategy & Synthetic Dataset Generation — Page 5
7. AI & Risk Scoring Methodology — Page 6
8. Evaluation Plan & Target Metrics — Page 6
9. Privacy, Security & Ethical Considerations — Page 7
10. Implementation Roadmap (Datathon Timeline) — Page 7
11. Six-Member Team Plan & Responsibilities — Page 8
12. Limitations & Future Scope — Page 8
13. References & Conclusion — Page 8

1. Executive Summary

In digital retail banking, Authorized Push Payment (APP) scams and psychological coercion—such as callers impersonating law enforcement officers ("digital arrest" scams), tax inspectors, or distressed family members—manipulate victims into voluntarily transferring their funds to fraudulent accounts. Because the genuine account holder inputs their legitimate PIN, password, and two-factor authentication, traditional banking fraud controls that look for account takeovers often fail to detect that the user is acting under duress or deception.

ScamGuard-Z is a lightweight, explainable transaction analysis prototype designed to address this vulnerability. Built specifically as a manageable, beginner-friendly project for a team of six students, ScamGuard-Z evaluates simulated transactions, detects combinations of suspicious factors (such as uncharacteristic amounts, unfamiliar recipients, transfer velocity, and high balance depletion ratios), categorizes transfers into **Low, Medium, or High** risk tiers, provides a clear human-readable rationale, and presents an educational safety warning encouraging users to pause and verify the transaction before execution.

The project uses 100% synthetic transaction data to protect privacy, prioritizes a clear rule-based scoring baseline, includes an optional machine-learning extension if time permits, and targets the **IBM LinuxONE Community Cloud (s390x)** to explore containerized deployment on enterprise mainframe architecture.

2. Problem Statement & Project Objectives

2.1 The Vulnerability of Authorized Coercion

Traditional banking fraud detection architectures are primarily engineered to prevent unauthorized access (e.g., credential stuffing, malware, or logins from unknown devices and unrecognized foreign IP addresses). However, social engineering attacks exploit the human element rather than technical system vulnerabilities. Scammers use high-pressure tactics, fabricated urgency, and fear to convince account holders to drain their savings accounts voluntarily. Because the legitimate account owner initiates and authorizes the transaction, conventional rule filters often treat the transfer as standard behavior.

2.2 Realistic Nuance: Suspicious Does Not Equal Scam

A foundational design principle of ScamGuard-Z is acknowledging that **not every unusual transaction is a scam**. Consumers frequently execute legitimate high-value or unfamiliar transfers—such as purchasing an automobile, making a home deposit, or sending money to a newly contracted service provider. Claiming that every anomaly is criminal would result in unacceptable user friction and false alarms. Therefore, ScamGuard-Z is not designed to unilaterally block accounts or prove psychological duress; rather, it serves as an **informative, explainable advisory check** that prompts users to pause and double-check when multiple risk indicators converge.

2.3 Core Project Objectives

- **Objective 1:** Construct a clean web simulation interface allowing users and evaluators to input mock payment details.
- **Objective 2:** Develop an explainable risk-scoring engine evaluating four observable transaction factors.
- **Objective 3:** Categorize risk into Low, Medium, and High tiers accompanied by clear explanations.
- **Objective 4:** Implement an empathetic safety intervention modal providing practical verification steps.
- **Objective 5:** Explore an optional Scikit-Learn machine-learning extension to benchmark against the rule-based engine.
- **Objective 6:** Explore packaging the backend into a container and deploying it on the IBM LinuxONE Community Cloud.

3. Proposed Solution & Core Prototype Features

To deliver a realistic, technically meaningful project within the 24-hour datathon timeframe, ScamGuard-Z focuses on a structured five-component prototype:

1. Simulation & Input Interface

A simple web form enabling users to test simulated transactions by inputting payment parameters: transfer amount, account balance prior to payment, transfer frequency in the last 24 hours, and whether the recipient is newly added or previously verified.

2. Explainable Risk-Scoring Engine

A deterministic baseline engine that calculates a composite Risk Score (0–100) based on transparent, inspectable mathematical weights across four key detection factors, ensuring total auditability and zero black-box obscurity.

3. Tiered Risk Assessment

Classifies each simulated transfer into actionable categories:

- **Low Risk (0–39):** Routine characteristics; standard proceed confirmation.
- **Medium Risk (40–69):** Isolated unusual attribute; gentle advisory note.
- **High Risk (70–100):** Multiple compounding risk factors; prominent safety warning.

4. Practical Safety Warning Modal

Instead of abrupt account locks, High-Risk transfers trigger a clear educational dialog:

- "*Are you on the phone with someone telling you to send this urgently?*"
- "*Did the caller claim to be police, tax staff, or bank fraud security?*"
- Practical verification checklist and one-click "Cancel & Review" option.

5. Optional Machine Learning Extension (Time & Data Permitting)

As an extension exercise, the team plans to implement a secondary machine-learning classifier (e.g., Logistic Regression or Random Forest using Python's Scikit-Learn) trained on synthetic transaction logs to compare predictive risk probabilities against the rule-based baseline.

3.1 Key Detection Factors Evaluated

The prototype evaluates four observable transaction signals commonly seen in social engineering scenarios:

Detection Factor	Description & Rationale	Simulated Risk Indicator
1. Atypical Transfer Amount	Compares transfer value against the account's synthetic historical median transfer size.	Transfer value exceeds 3× to 5× the account's historical average.
2. Unfamiliar Recipient Status	Evaluates whether the recipient account is previously established or newly created.	Beneficiary added within the past 24 hours with zero previous payment history.
3. Transfer Velocity (Frequency)	Monitors repeated outbound transfers occurring within a compressed timeframe.	Multiple outbound transfers executed within minutes or hours of each other.
4. Account Depletion Ratio	Measures what percentage of total liquid balance is being transferred out in one session.	Single transaction attempts to drain 75% to 95% of available funds.

4. System Architecture & Workflow

The architecture of ScamGuard-Z separates the frontend presentation, backend scoring services, and deployment target to ensure clean modularity and straightforward testing for beginner developers.

4.1 System Architecture Diagram

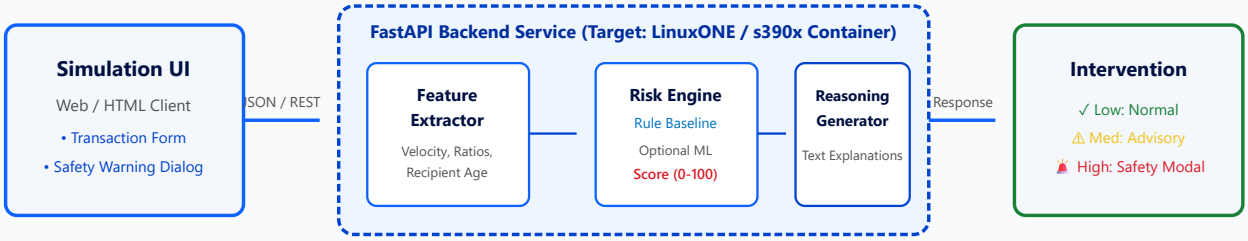


Figure 1: High-level ScamGuard-Z architecture from simulated input to risk output.

4.2 Risk Assessment Workflow



Figure 2: Sequential transaction evaluation and decision workflow.

5. Technology Stack & IBM Z / LinuxONE Exploration

Given that all six team members are beginners, the technology stack is intentionally selected for simplicity, active open-source documentation, and ease of debugging.

Technology	Component	Purpose & Beginner-Friendly Rationale
Python 3.x	Core Programming Language	Universal, highly readable language; rapid development; extensive library ecosystem for data processing and web services.
FastAPI	Backend REST API	Modern, lightweight web framework; straightforward routing; native data validation with Pydantic; auto-generated interactive documentation (/docs).
HTML5 / CSS3 / JS	Frontend Web Simulation	Zero framework overhead (no complex React/Angular build chains); easy to construct a clean transaction input form and alert modal.
Pandas & NumPy	Data Generation & Analysis	Industry standard for generating, structuring, and filtering synthetic tabular transaction data and calculating statistical baselines.
Scikit-Learn (Optional)	Machine Learning Extension	Accessible library for testing simple tabular models (Logistic Regression or Random Forest) if time allows during the hackathon.
IBM LinuxONE (Target)	Cloud Deployment Exploration	**Exploration Goal:** Package the FastAPI backend as a container (Docker/Podman) and explore deploying/validating on an enterprise s390x Linux virtual machine via LinuxONE Community Cloud.

> **Deployment Target Clarification**
>
> IBM Z / LinuxONE Community Cloud represents an **active exploration target** for our team to validate during the datathon, not a completed production deployment. Our objective is to learn how enterprise mainframes handle containerized microservices and compare local execution with s390x virtualized environments.

6. Data Strategy & Synthetic Dataset Generation

6.1 Synthetic Transaction Data as the Primary Source

Connecting to real bank APIs or collecting actual customer financial records poses severe privacy, legal, and ethical risks. Furthermore, genuine coercion and scam transactions are rarely labeled in public datasets due to bank confidentiality. Therefore, **ScamGuard-Z uses 100% synthetically generated transaction data as its primary foundation**.

Using a Python script, our team will generate synthetic datasets representing simulated account histories and payment requests:

- **Normal Simulated Patterns:** Routine transactions reflecting historical behaviors—modest transfer amounts (e.g., $20 to $150), recurring familiar recipients (e.g., utility providers, saved contacts), low transaction frequency (1–2 per day), and small account depletion ratios (<15%).
- **Suspicious Simulated Scenarios:** Parameterized to mimic common social engineering patterns—sudden high-value transfers ($2,000+), newly registered recipient accounts (<24 hours old), multiple transfers in rapid

succession, and transactions that liquidate >80% of available account funds.

6.2 Critical Distinction: Simulated Risk vs. Verified Fraud

> **Academic & Technical Rigor:** Labels in our dataset indicate *simulated risk profiles* based on synthetic behavioral parameters, **not ground-truth criminal convictions**. This distinction prevents circular reasoning and ensures evaluators understand the boundary between heuristic anomaly testing and true real-world fraud forensics.

6.3 Exploratory Public Datasets (Subject to Availability & Licensing)

The team will explore publicly accessible financial benchmarks, such as the **Kaggle IEEE-CIS Fraud Detection dataset**, solely as a reference point for feature distributions and tabular structures. Its use will remain strictly subject to dataset availability, download bandwidth constraints during the hackathon, and platform licensing terms.

7. AI & Risk Scoring Methodology

7.1 Rule-Based Baseline Engine (Primary Methodology)

For a team of beginners, starting with a transparent, deterministic rule-based engine ensures reliability, auditability, and immediate explainability. The engine assigns normalized sub-scores (0 to 1) to each of the four detection factors, multiplied by calibrated weights:

```
Total_Risk_Score = (W_amt × S_amt) + (W_recip × S_recip) + (W_vel × S_vel) + (W_dep × S_dep)

Proposed Baseline Weights (Sum = 100):
• W_amt (Atypical Amount) = 30 points
• W_recip (Unfamiliar Recipient) = 25 points
• W_vel (Transfer Velocity) = 25 points
• W_dep (Balance Depletion) = 20 points
```

7.2 Explainability Mechanism

Whenever a factor score exceeds its threshold, the engine attaches a human-readable explanation string to the API response. For instance, if a simulated transfer receives a score of 75 (High Risk), the response includes:

- "*Transfer amount is 4.2× higher than your 30-day average.*"
- "*Recipient account was added 3 hours ago with zero prior transfer history.*"
- "*This transaction transfers 85% of your total account balance.*"

7.3 Optional Machine-Learning Extension

If the baseline engine is completed ahead of schedule, the team will explore training a simple binary classification model (e.g., Scikit-Learn Random Forest or Logistic Regression) using an 80/20 train-test split on synthetic data. This extension is proposed as a comparative learning exercise to analyze whether tree-based decision boundaries align with our rule weights. *No pre-trained model is claimed at this stage.*

8. Evaluation Plan & Proposed Targets

To validate ScamGuard-Z objectively during testing, the team will execute test scenarios across simulated normal, borderline, and high-suspicion transactions. All performance targets are **proposed engineering goals**, not claimed results.

Evaluation Metric	Description & Purpose	Proposed Target
Precision (Simulated)	Measures what proportion of high-risk flags were truly intended suspicious scenarios.	Proposed Target: > 80% on synthetic test suite
Recall (Simulated)	Measures what proportion of synthetic high-duress scenarios were successfully flagged.	Proposed Target: > 85% on synthetic test suite
False-Positive Rate	Measures how often normal, everyday transactions are mistakenly flagged as high risk.	Proposed Target: < 15% to minimize user irritation
API Response Time	End-to-end latency from transaction submission to risk score and explanation return.	Proposed Target: < 500ms on simulated API calls

Explanation Clarity	Qualitative assessment: Are reasons easily understood by non-technical team members?	100% of high-risk outputs provide readable text

9. Privacy, Security & Ethical Considerations

Because financial applications handle sensitive human situations, ScamGuard-Z adheres strictly to ethical development practices appropriate for a student hackathon:

- **Synthetic Data Exclusivity:** The prototype never connects to live banking rails, real bank accounts, or real payment credentials. All names, balances, and account numbers are fictional.
- **Data Minimization:** The engine processes only essential transaction parameters (amount, timestamp delta, recipient status, balance percentage) without capturing or storing personal identifying information (PII).
- **Non-Coercive User Agency:** The system does not freeze accounts permanently. It introduces an intentional cognitive speed-bump, respecting customer autonomy while providing critical safety advice.
- **Absence of Unsubstantiated Claims:** The project does not claim compliance with PCI-DSS or GDPR certification, but adopts privacy-by-design principles suited for a prototype.

10. Implementation Roadmap (24-Hour Datathon Timeline)

Phase & Hours	Key Milestone	Deliverables & Verification
Phase 1: Setup & Data (Hours 0–4)	Environment preparation & synthetic dataset creation	Git repository initialized; Python synthetic generation script written producing 500+ mock records.
Phase 2: Core Engine (Hours 4–8)	Rule-based risk scoring & explanation logic	Python functions implementing the four detection factor weights and explanation text generators.
Phase 3: Backend API (Hours 8–12)	FastAPI web service integration	Working REST endpoints (`/evaluate-transaction`); interactive testing via Swagger UI (`/docs`).
Phase 4: Simulation UI (Hours 12–16)	Web interface & safety modal dialog	Interactive form allowing transaction input; dynamic popup displaying Low/Med/High alerts.
Phase 5: Testing & ML (Hours 16–20)	System testing & optional ML exploration	Test suite executed against test scenarios; optional Scikit-Learn model exploration if time permits.
Phase 6: LinuxONE & Pitch (Hours 20–24)	LinuxONE target exploration & demo video	Attempt container deployment on LinuxONE Community Cloud; record 3-minute video pitch & finalize repo.

11. Six-Member Team Plan & Responsibilities

To ensure all six beginner team members have clear, manageable responsibilities and contribute meaningfully to the project, roles are structured as follows:

Member Role	Primary Focus Area	Key Responsibilities	Deliverables
Member 1 (Team Lead)	Project Coordination & Video Pitch	Manages task timelines, organizes documentation, scripts and coordinates the 3-minute presentation video.	Project README, presentation script, final video.

Member 2	Synthetic Data & Scenarios	Develops the Python script to create synthetic normal and suspicious transaction records; documents test cases.	`generate_data.py`, CSV test datasets.
Member 3	Risk Engine & Rules	Implements the core scoring logic, mathematical weight distribution, and text explanation generator functions.	`risk_engine.py`, rule logic unit tests.
Member 4	Backend API (FastAPI)	Develops the FastAPI endpoints, handles JSON request/response validation with Pydantic, connects risk engine.	`main.py` API service, Swagger endpoint docs.
Member 5	Simulation UI & Modals	Designs the transaction input web form, formats risk tier badges, and builds the safety warning intervention modal.	`index.html`, `style.css`, frontend scripts.
Member 6	LinuxONE & Testing QA	Explores containerizing the app (Dockerfile), tests endpoints on LinuxONE Community Cloud, logs test metrics.	`Dockerfile`, test log summary, LinuxONE notes.

Shared Team Responsibility: All six members collaborate during Hours 16–20 on integrated end-to-end testing, slide review, and peer rehearsal.

12. Limitations & Future Scope

12.1 Prototype Limitations

- **Cannot Prove True Coercion:** A software algorithm operating solely on transaction metadata cannot determine a user's psychological state or verify what is happening off-device on a telephone call.
- **Potential for False Positives:** Legitimate urgent transactions (e.g., medical emergency bill payment) share attributes with coercion fraud (high amounts, urgent timing), leading to advisory warnings on legitimate transfers.
- **Does Not Replace Bank Systems:** ScamGuard-Z is an experimental educational concept designed to augment human awareness, not replace professional fraud departments, human investigators, or established anti-money laundering (AML) controls.

12.2 Future Scope (Post-Datathon Opportunities)

- **Multi-Party Verification:** Exploring opt-in "trusted contact" verification where high-risk transactions notify a pre-selected family member for advisory confirmation.
- **Enterprise Mainframe Integration:** Deepening technical validation on IBM Z systems to examine real-time transactional throughput and hardware-level cryptographic services.
- **Ethical Behavioral Biometrics:** Investigating whether ethical, client-side input hesitation patterns (e.g., typing pauses during payee entry) can add non-intrusive safety signals.

13. References

1. **Payment Systems Regulator (PSR), UK.** (2023). *Authorised Push Payment (APP) Scams: Policy Statement and Performance Metrics*. Available at: https://www.psr.org.uk/
2. **Global Anti-Scam Alliance (GASA).** (2023). *The Global State of Scams Report: Consumer Vulnerability and Social Engineering Trends*. Available at: https://www.gasa.org/
3. **FastAPI Documentation.** (2024). *High Performance Python Web Framework*. Tiangolo. Available at: https://fastapi.tiangolo.com/
4. **Pedregosa, F., et al.** (2011). *Scikit-learn: Machine Learning in Python*. Journal of Machine Learning Research, 12, pp. 2825–2830.
5. **IBM Corporation.** (2024). *LinuxONE Community Cloud Documentation and Fast Start Guides*. IBM Z Community. Available at: https://community.ibm.com/zsystems/l1cc/

14. Conclusion

ScamGuard-Z approaches the pervasive challenge of social engineering and coercion scams through a realistic, human-centric design. By acknowledging that traditional security filters often miss authorized transactions conducted under duress, the project introduces a transparent, explainable scoring engine that empowers users with a crucial moment of reflection.

Engineered as an achievable and technically meaningful project for a team of six beginner undergraduate students, ScamGuard-Z establishes a solid foundation using Python, FastAPI, and synthetic data, while providing an exciting pathway to explore enterprise mainframe deployment on the IBM LinuxONE Community Cloud.